

GUÍA DE LABORATORIO

SOURCING MANAGER · ASP.NET CORE MVC

Laboratorio 1

Configuración del proyecto y CRUD de Proveedores

Stack	ASP.NET Core MVC + EF Core
Base de datos	SQLite (compatible con SQL Server)
Nivel	Básico → Intermedio
Duración estimada	2 - 3 horas
Prerrequisitos	.NET SDK 8, C# básico

Objetivo del laboratorio

Al finalizar, tendrás una aplicación MVC funcionando con:

- Modelo de datos inicial (`Proveedor` , `CategoriaProveedor`)
- Base de datos creada vía migraciones (SQLite)
- CRUD completo (Crear, Leer, Actualizar, Eliminar) de Proveedores
- Datos de ejemplo precargados

0 Crear el proyecto

```
dotnet new mvc -n SourcingManager
cd SourcingManager
```

Verifica que corre antes de tocar nada:

```
dotnet run
```

Abre la URL que te indique la consola (usualmente `https://localhost:5001`). Deberías ver la página de bienvenida por defecto.

✓ Checkpoint 1

El proyecto compila y corre sin errores.

1 Instalar paquetes necesarios

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

Si alguien prefiere SQL Server en vez de (o además de) SQLite:

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

2 Crear los modelos

Crea la carpeta `Models` (si no existe) y agrega:

Models/CategoriaProveedor.cs

```
using System.ComponentModel.DataAnnotations;

namespace SourcingManager.Models
{
    public class CategoriaProveedor
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "El nombre es obligatorio")]
        [StringLength(80)]
        public string Nombre { get; set; }

        public ICollection<Proveedor>? Proveedores { get; set; }
    }
}
```

Models/Proveedor.cs

```
using System.ComponentModel.DataAnnotations;

namespace SourcingManager.Models
{
    public class Proveedor
    {
        public int Id { get; set; }
    }
}
```

```

[Required(ErrorMessage = "El nombre es obligatorio")]
[StringLength(120)]
public string Nombre { get; set; }

[Required]
[StringLength(20)]
public string RucONit { get; set; }

[EmailAddress(ErrorMessage = "Email inválido")]
public string? Email { get; set; }

[Phone]
public string? Telefono { get; set; }

public string? Direccion { get; set; }

[Required(ErrorMessage = "Selecciona una categoría")]
public int CategoriaProveedorId { get; set; }
public CategoriaProveedor? CategoriaProveedor { get; set; }
}
}

```

Pregunta guía

¿Por qué `CategoriaProveedor` está marcada con `?` (nullable) en `Proveedor`, pero `CategoriaProveedorId` no? (Pista: una es la clave foránea obligatoria, la otra es la propiedad de navegación que EF resuelve después).

3 Crear el DbContext

Creas la carpeta `Data` y agregas:

Data/SourcingDbContext.cs

```

using Microsoft.EntityFrameworkCore;
using SourcingManager.Models;

namespace SourcingManager.Data
{
    public class SourcingDbContext : DbContext
    {
        public SourcingDbContext(DbContextOptions<SourcingDbContext> options)
            : base(options) { }

        public DbSet<Proveedor> Proveedores { get; set; }
        public DbSet<CategoriaProveedor> CategoriasProveedor { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Datos semilla
            modelBuilder.Entity<CategoriaProveedor>().HasData(
                new CategoriaProveedor { Id = 1, Nombre = "Tecnologia" },
                new CategoriaProveedor { Id = 2, Nombre = "Materia Prima" },
                new CategoriaProveedor { Id = 3, Nombre = "Logística" }
            );
        }
    }
}

```

4 Configurar la conexión

`appsettings.json` — agrega la cadena de conexión:

```

{
  "ConnectionStrings": {
    "SqliteConnection": "Data Source=sourcing.db"
  },
  "Logging": { ... },
  "AllowedHosts": "*"
}

```

Program.cs — antes de `var app = builder.Build();`:

```
using Microsoft.EntityFrameworkCore;
using SourcingManager.Data;

builder.Services.AddDbContext<SourcingDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("SqliteConnection"));
```

5 Crear la migración y la base de datos

```
dotnet ef migrations add InicialProveedores
dotnet ef database update
```

✓ Checkpoint 2

Debe aparecer un archivo `sourcing.db` en la raíz del proyecto. Ábrelo con "DB Browser for SQLite" (opcional) y confirma que existen las tablas `Proveedores` y `CategoriasProveedor` con las 3 categorías semilla.

Si algo falla

Revisa que `Microsoft.EntityFrameworkCore.Tools` esté instalado y que no haya errores tipográficos en el namespace del `DbContext`.

6 Generar el CRUD (scaffolding)

Instala la herramienta de scaffolding si no la tienes:

```
dotnet tool install --global dotnet-aspnet-codegenerator
dotnet add package Microsoft.VisualStudio.Web.CodeGeneration.Design
```

Genera el controller + vistas para `Proveedor` :

```
dotnet aspnet-codegenerator controller -name ProveedoresController -m Proveedor -dc SourcingDbContext -outDir Controllers --
relativeFolderPath Views/Proveedores --referenceScriptLibraries
```

Esto crea:

- `Controllers/ProveedoresController.cs`
- `Views/Proveedores/Index.cshtml` , `Create.cshtml` , `Edit.cshtml` , `Details.cshtml` , `Delete.cshtml`

✓ Checkpoint 3

Corre `dotnet run` y navega a `/Proveedores`. Deberías ver una tabla vacía con botón "Create New".

7 Probar el CRUD manualmente

1. Crea 3 proveedores de prueba (usa las categorías semilla)
2. Edita uno y cambia su teléfono
3. Verifica que las validaciones funcionen: intenta crear un proveedor sin nombre o con un email inválido — debe mostrar el mensaje de error en la vista, no un error 500
4. Elimina uno de los proveedores de prueba

✓ Checkpoint 4

Las 4 operaciones (crear, listar, editar, eliminar) funcionan sin errores y las validaciones se muestran correctamente en el formulario.

8 Mejora del scaffolding: mostrar el nombre de categoría

Por defecto, el scaffolding en `Index.cshtml` puede mostrar `CategoriaProveedorId` en vez del nombre. En el controller, revisa que el `Index` incluya la navegación:

```
public async Task<IActionResult> Index()
{
    var proveedores = _context.Proveedores.Include(p => p.CategoriaProveedor);
    return View(await proveedores.ToListAsync());
}
```

Y en `Index.cshtml` , cambia la columna correspondiente para mostrar `item.CategoriaProveedor.Nombre` en vez del `Id`.

Entregable del laboratorio

1. El código fuente completo (o link al repositorio)
2. Captura de pantalla de la lista de proveedores con al menos 3 registros
3. Captura del formulario de creación mostrando un mensaje de validación en acción
4. Respuesta corta (2-3 líneas): ¿qué pasaría si `CategoriaProveedorId` no fuera `Required` ? ¿Qué error de integridad podría ocurrir?

Checklist final

- ☐ Proyecto corre sin errores
- ☐ Migraciones aplicadas y base de datos con datos semilla
- ☐ CRUD de Proveedor 100% funcional
- ☐ Validaciones visibles en el formulario
- ☐ Index muestra el nombre de la categoría, no el Id numérico

Siguiente laboratorio

El Laboratorio 2 partirá de este proyecto para agregar `Producto`, `SolicitudCotizacion` y la relación N-N entre ambos (selección múltiple de productos en un formulario). Guarda tu proyecto — lo seguirás usando.